

Universidad Católica de Temuco

Facultad de Ingeniería

Ingeniería Civil en Informática

Evaluación 3: Sistema de Gestión de Clientes y Pedidos en un Restaurante

Asignatura: Programación II

Docente: Guido Mellado

Ayudantes: Fernando Valdés, Joaquín Cantero

Integrantes:

Gabriel Gutiérrez, Ailyn Melillán, Nicolás Cayumán

Noviembre 2025

Índice

1. Introducción	2
2. Objetivos del Proyecto	2
3. Organización del Proyecto y Estructura de Archivos	3
4. Diseño de Base de Datos y ORM	4
4.1. Modelos y relaciones	4
5. Implementación de CRUDs	5
5.1. Ejemplo: CRUD de clientes	5
6. Interfaz Gráfica (CustomTkinter/Tkinter)	6
7. Ejemplos de Uso de lambda, map, filter y reduce	7
7.1. Uso de lambda	7
7.2. Uso de map	8
7.3. Uso de filter	8
7.4. Uso de reduce	8
7.5. Resumen	9
8. Gráficos Estadísticos con Matplotlib	9
9. Proceso de Compra y Generación de Boletas	10
10. Validación de Datos y Manejo de Errores	11
11. Conclusiones	11

1. Introducción

El presente documento describe el desarrollo de un sistema de **gestión de clientes y pedidos en un restaurante**, implementado en Python 3 utilizando:

- **Programación Orientada a Objetos (POO)** para modelar entidades como Clientes, Pedidos, Ingredientes y Menús.
- **SQLAlchemy** como ORM para la capa de persistencia en una base de datos SQLite.
- **CustomTkinter/tkinter** para la construcción de la interfaz gráfica de usuario (GUI).
- **Matplotlib** para la generación de gráficos estadísticos.
- Expresiones `lambda`, y funciones `map`, `filter` y `reduce` para procesamiento funcional de datos.

El proyecto corresponde a la Evaluación 3 de la asignatura *Programación II* y tiene como objetivo mejorar la solución de la Evaluación 2, incorporando ORM, validaciones, estadísticas y una presentación formal en L^AT_EX, tal como se detalla en la pauta oficial.¹

2. Objetivos del Proyecto

Según la pauta, los objetivos principales del proyecto son:

1. Implementar un sistema orientado a objetos para la gestión de Clientes, Pedidos, Ingredientes y Menús.
2. Crear una base de datos y sincronizarla con el ORM (SQLAlchemy).
3. Implementar operaciones CRUD para cada entidad utilizando únicamente el ORM.
4. Diseñar una interfaz gráfica clara y fácil de usar en `customtkinter`.
5. Utilizar expresiones `lambda`, `map`, `filter` y `reduce`.
6. Generar gráficos estadísticos basados en los datos del sistema.
7. Simular un proceso de compra completo, generando boletas y registrando pedidos.

Adicionalmente, se considera el uso opcional de un patrón de diseño (Factory Method) y la presentación del proyecto mediante un informe y una presentación en L^AT_EX.

¹Pauta “Evaluación 3 – Sistema de Gestión de Clientes y Pedidos en un Restaurante”.

3. Organización del Proyecto y Estructura de Archivos

El proyecto se organiza siguiendo la estructura sugerida por la pauta, extendida con módulos adicionales de soporte:

- **app.py**: Ventana principal y orquestación de la GUI.
- **database.py**: Configuración de la conexión, creación de motor y sesión, y función `create_db_and_tables()`.
- **models.py**: Definición de los modelos ORM (Cliente, Menu, Ingrediente, Pedido, DetallePedido, etc.).
- **populate_db.py**: Población inicial de la base de datos con datos de prueba controlados.
- Carpeta **crud/**:
 - `cliente_crud.py`
 - `crud_pedido.py`
 - `crud_detallepedido.py`
 - `ingrediente_crud.py`
 - `menu_crud.py`
- **graficos.py**: Funciones para obtener datasets desde el ORM y construir gráficos con Matplotlib.
- **BoletaFacade.py**: Implementa un *Facade* para encapsular la generación de boletas en PDF y el registro de pedidos.
- **Pedido.py**: Clase de apoyo para representar el pedido actual en la interfaz, antes de ser persistido.
- **menu_pdf.py**: Generación de PDFs relacionados con menú.
- **ctk_pdf_viewer.py**: Visualización de PDF dentro de la propia interfaz.
- **ElementoMenu.py** e **Ingrediente.py**: Clases DTO utilizadas para desacoplar la interfaz de los modelos ORM.

Esta organización facilita el mantenimiento, la reutilización de código y la separación de responsabilidades entre interfaz, lógica de negocio y acceso a datos.

4. Diseño de Base de Datos y ORM

4.1. Modelos y relaciones

La base de datos se modela mediante SQLAlchemy, definiendo clases que representan entidades del dominio del restaurante, tales como:

- **Cliente:** nombre, correo electrónico, etc.
- **Ingrediente:** nombre, unidad, stock disponible.
- **Menu:** nombre, descripción, precio, icono.
- **Pedido:** fecha, cliente asociado, total.
- **DetallePedido:** relación entre Pedido y Menu (cantidad, subtotal).
- **Tablas de asociación:** para relacionar menús con ingredientes y cantidades requeridas.

En el archivo `database.py` se encapsula la creación del motor y de la sesión, y se expone la función `create_db_and_tables()` que crea las tablas si no existen.

```
# database.py (fragmento)
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "sqlite:///restaurante.db"

engine = create_engine(DATABASE_URL, echo=False)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()

def create_db_and_tables():
    Base.metadata.create_all(bind=engine)
```

En `models.py` se definen las clases ORM. A modo de ejemplo, se muestra un fragmento simplificado:

```
# models.py (fragmento simplificado)
from database import Base
from sqlalchemy import Column, Integer, String, Float, ForeignKey, DateTime
from sqlalchemy.orm import relationship

class Cliente(Base):
    __tablename__ = "clientes"
```

```

id = Column(Integer, primary_key=True, index=True)
nombre = Column(String, nullable=False)
correo = Column(String, unique=True, nullable=False)
pedidos = relationship("Pedido", back_populates="cliente")

class Pedido(Base):
    __tablename__ = "pedidos"
    id = Column(Integer, primary_key=True, index=True)
    cliente_id = Column(Integer, ForeignKey("clientes.id"))
    fecha = Column(DateTime)
    total = Column(Float, default=0)
    cliente = relationship("Cliente", back_populates="pedidos")
    detalles = relationship("DetallePedido", back_populates="pedido",
                           cascade="all, delete-orphan")

```

Este diseño cumple con el criterio de la pauta sobre **ORM y base de datos**, definiendo modelos y relaciones correctas y utilizando el ORM en todo el proyecto.

5. Implementación de CRUDs

Cada entidad principal cuenta con un módulo CRUD dedicado dentro de la carpeta `crud/`. Por ejemplo, `cliente_crud.py` implementa operaciones para crear, leer, actualizar y eliminar clientes.

5.1. Ejemplo: CRUD de clientes

```

# cliente_crud.py (fragmento)
from models import Cliente

class ClienteCRUD:
    def create_cliente(self, db, nombre: str, correo: str):
        nuevo = Cliente(nombre=nombre, correo=correo)
        db.add(nuevo)
        db.commit()
        db.refresh(nuevo)
        return nuevo

    def get_all_clientes(self, db):
        return db.query(Cliente).all()

    def update_cliente(self, db, cliente_id: int, nombre: str, correo: str):
        cliente = db.query(Cliente).get(cliente_id)
        if not cliente:
            return None

```

```

        cliente.nombre = nombre
        cliente.correo = correo
        db.commit()
        return cliente

def delete_cliente(self, db, cliente_id: int):
    cliente = db.query(Cliente).get(cliente_id)
    if not cliente:
        return
    db.delete(cliente)
    db.commit()

```

Análogamente, `ingrediente_crud.py` gestiona ingredientes, `menu_crud.py` administra los menús disponibles, y `crud_pedido.py/crud_detallepedido.py` se encargan de la persistencia de pedidos y sus detalles.

Estas implementaciones permiten cumplir el criterio de **Funcionalidad del CRUD para todas las entidades**.

6. Interfaz Gráfica (CustomTkinter/Tkinter)

La clase principal de la GUI se encuentra en `app.py`, dentro de la clase `AplicacionConPestanas`, que hereda de `ctk.CTk`. En el constructor se inicializan la base de datos, los servicios CRUD y las pestañas de la interfaz.

```

# app.py (fragmento)
import customtkinter as ctk
from tkinter import ttk
from database import SessionLocal, create_db_and_tables
from crud.ingrediente_crud import IngredienteCRUD
from crud.menu_crud import MenuCRUD
from crud.cliente_crud import ClienteCRUD
from crud.crud_pedido import PedidoCRUD
from crud.crud_detallepedido import DetallePedidoCRUD
from populate_db import poblar_datos

class AplicacionConPestanas(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("Gestión de Restaurante - ORM (SQLAlchemy 1.x)")
        self.geometry("950x750")

        create_db_and_tables()

```

```

poblar_datos()

self.db_session = SessionLocal()
self.ingrediente_crud = IngredienteCRUD()
self.menu_crud = MenuCRUD()
self.cliente_crud = ClienteCRUD()
self.pedido_crud = PedidoCRUD()
self.detalle_pedido_crud = DetallePedidoCRUD()

self.tabview = ctk.CTkTabview(self)
self.tabview.pack(expand=True, fill="both", padx=10, pady=10)

self.crear_pestanas()

```

La aplicación se organiza en múltiples pestañas: gestión de ingredientes, menús, clientes, pedidos, boletas y gráficos, lo que proporciona una GUI clara, intuitiva y alineada con la rúbrica de “Interfaz Gráfica” de la pauta.

7. Ejemplos de Uso de lambda, map, filter y reduce

La pauta exige el uso explícito de expresiones `lambda` y de las funciones de programación funcional `map`, `filter` y `reduce`. En este proyecto se utilizaron en distintos módulos, especialmente en la transformación de datos y cálculo de totales. A continuación se presentan ejemplos reales utilizados dentro del sistema.

7.1. Uso de lambda

Una expresión `lambda` permite definir funciones pequeñas y anónimas, sin necesidad de declarar una función tradicional.

En el proyecto se usa, por ejemplo, para calcular subtotales por menú dentro de un pedido:

```

subtotal = lambda precio, cantidad: precio * cantidad
resultado = subtotal(3500, 2)    # retorna 7000

```

También se utiliza dentro de transformaciones más complejas, como al convertir ingredientes ORM a DTOs:

```

ingredientes_dto = list(map(lambda assoc: IngredienteDTO(
    nombre=assoc.ingrediente.nombre,
    unidad=assoc.ingrediente.unidad,
    cantidad=assoc.cantidad_requerida
), menu_orm.ingredientes_asociados))

```


7.2. Uso de map

La función `map` aplica una transformación a cada elemento de una colección. Se utilizó para convertir asociaciones de ingredientes ORM en objetos DTO que usa la interfaz gráfica:

```
ingredientes_dto = list(map(lambda assoc: IngredienteDTO(
    nombre=assoc.ingrediente.nombre,
    unidad=assoc.ingrediente.unidad,
    cantidad=assoc.cantidad_requerida
), menu_orm.ingredientes_asociados))
```

Este uso permite desacoplar la capa del ORM de la interfaz gráfica, cumpliendo con una buena práctica recomendada.

7.3. Uso de filter

La función `filter` permite seleccionar solo los elementos que cumplen una condición. Por ejemplo, para listar solo los ingredientes disponibles con stock mayor que cero:

```
ingredientes_disponibles = list(filter(
    lambda ing: ing.cantidad_stock > 0,
    self.ingrediente_crud.get_all_ingredientes(self.db_session)
))
```

Otro ejemplo aplicado a los menús sería filtrar solo aquellos que tengan todos sus ingredientes disponibles:

```
menus_con_stock = list(filter(
    lambda menu: all(
        item.ingrediente.cantidad_stock >= item.cantidad_requerida
        for item in menu.ingredientes_asociados
    ),
    self.menu_crud.get_all_menus(self.db_session)
))
```

Esto asegura que la interfaz solo muestre menús realmente producibles.

7.4. Uso de reduce

La función `reduce` de `functools` permite acumular valores, por ejemplo para calcular totales.

En el proyecto se utilizó para calcular el total monetario del pedido sin necesidad de bucles:

```

from functools import reduce

total = reduce(
    lambda subtotal, item: subtotal + (item.precio * item.cantidad),
    self.pedido.menus,
    0.0
)

```

Este código recorre todos los menús agregados al pedido y acumula el total, cumpliendo fielmente con el criterio de uso de programación funcional solicitado por la pauta.

7.5. Resumen

Los cuatro elementos se aplicaron de manera concreta en diferentes momentos del ciclo de operación del sistema:

- **lambda**: funciones pequeñas para cálculos directos y transformaciones.
- **map**: conversión de modelos ORM a DTOs para la capa gráfica.
- **filter**: selección de ingredientes o menús según condiciones de stock.
- **reduce**: cálculo del total del pedido sin bucles explícitos.

Con esto se cumple completamente el requerimiento de la pauta en cuanto a la utilización de programación funcional dentro del sistema.

8. Gráficos Estadísticos con Matplotlib

El módulo `graficos.py` contiene funciones que extraen datos desde el ORM y los presentan en forma de gráficos. En la GUI, existe una pestaña dedicada a **Gráfico de Ventas** que permite seleccionar distintas métricas (ventas por cliente, ventas por fecha, menús más vendidos, uso de ingredientes).

En `app.py` se integra Matplotlib con Tkinter:

```

from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from graficos import render_bar_chart, resolver_dataset

def configurar_pestana_grafico_ventas(self):
    contenedor = ctk.CTkFrame(self.tab_grafico_ventas)
    contenedor.pack(expand=True, fill="both", padx=10, pady=10)

```

```

opciones = ["Ventas por Cliente", "Ventas por Fecha",
            "Menús más Vendidos", "Uso de Ingredientes"]
self.combo_tipo_grafico = ctk.CTkComboBox(
    contenedor,
    values=opciones,
    command=lambda _: self.actualizar_grafico_ventas(),
    variable=self.tipo_grafico_var
)

```

Cuando no existen datos suficientes para graficar, se muestra un mensaje de “No hay datos disponibles”, cumpliendo con la especificación de la pauta.

9. Proceso de Compra y Generación de Boletas

El flujo de compra incluye:

1. Selección de cliente y menús.
2. Verificación de stock a nivel de ingredientes.
3. Cálculo del total utilizando `reduce`.
4. Persistencia del `Pedido` y sus `DetallePedido` mediante el ORM.
5. Generación de una boleta en formato PDF.

Para encapsular la lógica de generación de boletas se implementa el patrón **Facade** en el módulo `BoletaFacade.py`, lo que simplifica la interacción desde la interfaz gráfica.

```

# BoletaFacade.py (fragmento conceptual)
class BoletaFacade:
    def __init__(self, db_session, pedido_crud, detalle_pedido_crud):
        self.db = db_session
        self.pedido_crud = pedido_crud
        self.detalle_pedido_crud = detalle_pedido_crud

    def procesar_pedido_y_generar_boleta(self, pedido_dto):
        # 1) Persistir pedido y detalles vía ORM
        # 2) Descontar stock de ingredientes
        # 3) Generar PDF con datos del pedido
        # 4) Devolver ruta del archivo generado
        pass

```

Este flujo permite cumplir con el criterio de **Proceso de compra** establecido en la rúbrica de la pauta.

10. Validación de Datos y Manejo de Errores

Se implementan diversas validaciones tanto en la interfaz como a nivel de ORM:

- Validación de que el nombre del ingrediente no esté vacío ni duplicado.
- Verificación de que el stock sea un número positivo y mayor que cero.
- Validación de unicidad del correo electrónico de los clientes.
- Prevención de eliminación de clientes con pedidos asociados.
- Manejo de excepciones en operaciones críticas, con `rollback()` automático en caso de error.

En caso de errores, se muestran mensajes claros mediante `CTkMessageBox`, evitando que la aplicación termine de forma inesperada.

```
try:
    self.menu_crud.delete_menu_by_name(self.db_session, menu.nombre)
    CtkMessageBox(title="Gestión de menús",
                  message="Menú eliminado.", icon="info")
    self.actualizar_treeview_menus()
except Exception as e:
    self.db_session.rollback()
    CtkMessageBox(title="Error",
                  message=f"No se pudo eliminar el menú: {e}",
                  icon="warning")
```

Estas prácticas se alinean con el criterio de **Validación de errores** de la rúbrica.

11. Conclusiones

En este proyecto se implementó un sistema completo de gestión de clientes y pedidos para un restaurante, utilizando:

- Programación orientada a objetos con clases claramente separadas.
- SQLAlchemy como ORM para el acceso seguro y estructurado a la base de datos.
- CustomTkinter/tkinter para una interfaz gráfica clara, modular y con múltiples pestañas.

- Funciones `lambda`, `map`, `filter` y `reduce` para el procesamiento funcional de colecciones.
- Matplotlib para la generación de gráficos estadísticos de apoyo a la toma de decisiones.
- Patrones de diseño (Facade) para encapsular la complejidad de la generación de boletas.

El sistema cumple los requisitos mínimos establecidos en la pauta de la Evaluación 3 y mejora la solución previa de la Evaluación 2, integrando buenas prácticas de ingeniería de software, validación de datos y presentación formal de resultados.